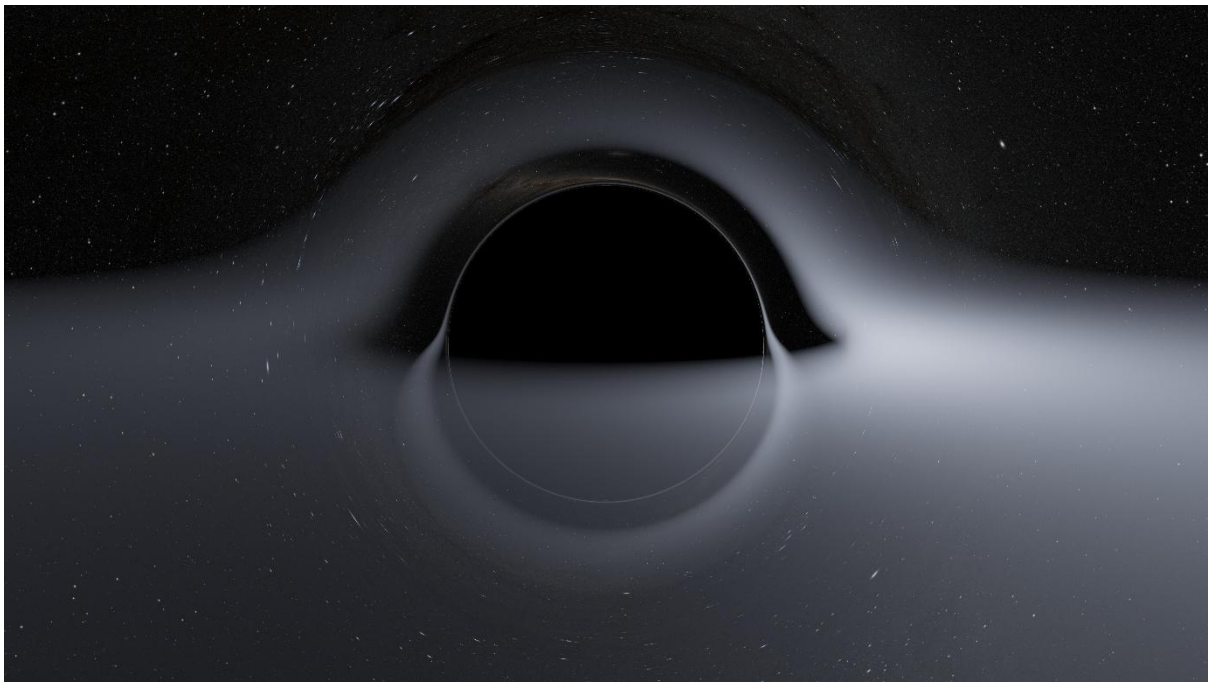
CREST Gold submission

Ray tracing a Kerr Black Hole

By Srikanth Reddy Mettu



Criteria Map

CREST Gold criterion	Where it is covered	Page / done
1.1 Clear aim, broken into objectives	§2.1, §2.2	8
1.2 Wider purpose explained	§1.2, §5.4	5,24
1.3 Range of approaches identified	§2.3	8
1.4 Plan described, with reasons	§2.3	8
1.5 Time planned and organised	Appendix A (timeline)	30
2.1 Good use of materials and people	§2.4	10
2.2 Background research + referencing	§1.3, References	6,27
3.1 Logical conclusions + implications	§5.1, §5.4	23,24
3.2 How decisions affected the outcome	§2.3, §3, §4, §5.1	8, 11–21, 23
3.3 What was learnt + reflection	§6 and Profile Form	24
4.1 Understanding of the science	§1.3, §4, §5	6, 17–21, 23–24
4.2 Ethics and safety in decisions	§7	25
4.3 Creative thinking	§4, §5.3	17–21, 24
4.4 Problems identified and overcome	§3.3, §3.7, §3.8	12,15,17
4.5 Project explained clearly	Whole document	

Contents

Abstract 4
1. Introduction 5
1.1 Why I chose this project 5
1.2 Why it matters 5
1.3 Background and literature review 6
2. Project Aims and Planning 8
2.1 Aim and success criteria 8
2.2 Objectives and milestones 8
2.3 Approaches considered & Plan 8
2.4 Materials and people 10
3. Methodology 11
3.1 Code architecture 11
3.2 Equations of Motion 12
3.3 Numerical Integration 12
3.4 Accretion Disk model 14
3.5 Relativistic Effects 14
3.6 Rendering 15
3.7 Camera Initialisation 15
3.8 GPU acceleration and frame generation 17
4. Results and Discussion 17
4.1 Validation against Theory 17
Drift in Conserved Quantities 17
Comparison of Kerr hole shadow against Bardeen's Analytical Critical Curve 18
4.2 Performance 20
GPU vs CPU 20
RK-4 vs RK-45 20
4.3 Parameter Study 21
5. Conclusion 23

5.1 Were the aims met? 23
5.2 Limitations 23
5.3 Further work 24
5.4 Implications for the wider world 24
6. Reflection 24
7. Ethics, safety and authenticity 25
7.1 Scientific communication ethics and safety 25
7.2 Licensing and Attribution 25
7.3 Environmental cost 25
7.4 Authenticity and tool disclosure 25
8. References 27
9. Glossary 29
10. Appendices 30
Appendix A: Project Timeline 30
Planned vs Actual project timeframe 32
Appendix B: Kerr metric and equations of motion 33
Appendix C: Butcher Tableau for Dormand Prince Numerical Integration 34

Abstract

Near a rotating (Kerr) black hole, light follows curved trajectories known as geodesics, which are hard to compute directly as their differential equations of motion are not practical to evaluate in closed form. I built a relativistic ray-tracing program in Python of the Kerr metric that uses the adaptive Dormand prince (RK-45) method to numerically integrate the null geodesics in Boyer-Lindquist coordinates with Mino time and parallelises the code across GPU cores. It matched the analytic Bardeen critical curve to 10^{-3} order. This code could be used for spreading public understanding of general relativity and is accessible enough to be run on a home laptop.

The link to the final rendering is available here.

1. Introduction

1.1 Why I chose this project

After reading up about General relativity online, I was intrigued by the interesting phenomena that occur near a black hole, such as the apparent bending of light rays near it, resulting in the formation of Einstein rings and the iconic accretion disk shape around it. To explore this further, I built a ray tracing simulation of the Kerr metric.

1.2 Why it matters

I believe the most important aspect of simulations like this is public understanding and education. Imaging black holes is one of the few aspects of general relativity accessible to non-specialists in the field (TOI World Desk, 2024). However, the issue arises that most renders available online are artistic rather than scientifically accurate and may convey the wrong idea. Hence, I want to demonstrate more realistic simulations that can both be captivating and teach the audience about the geometry of spacetime. The Event Horizon Telescope (EHT) has successfully imaged real black holes such as M87* and Sagittarius A*, generating significant public interest. It is hence important to be able to explain these results with the help of simulations. (EHT Collaboration, 2019 & 2022)

This code runs on a home laptop with a regular GPU, making it more accessible compared to research level programs which often require significantly higher computational power. Hence, students across the world could run it or improve it by accounting for other relativistic effects such as aberration on a home computer.

1.3 Background and literature review

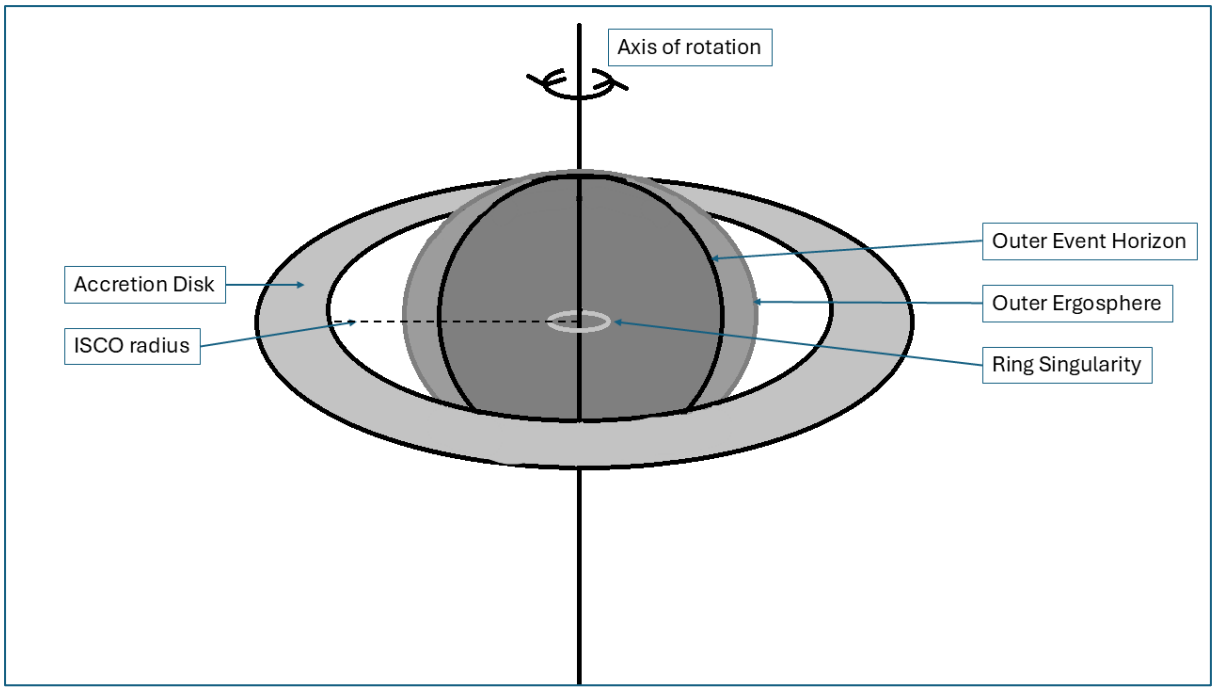


Figure 1: Schematic representation of the Kerr black hole with important features labelled

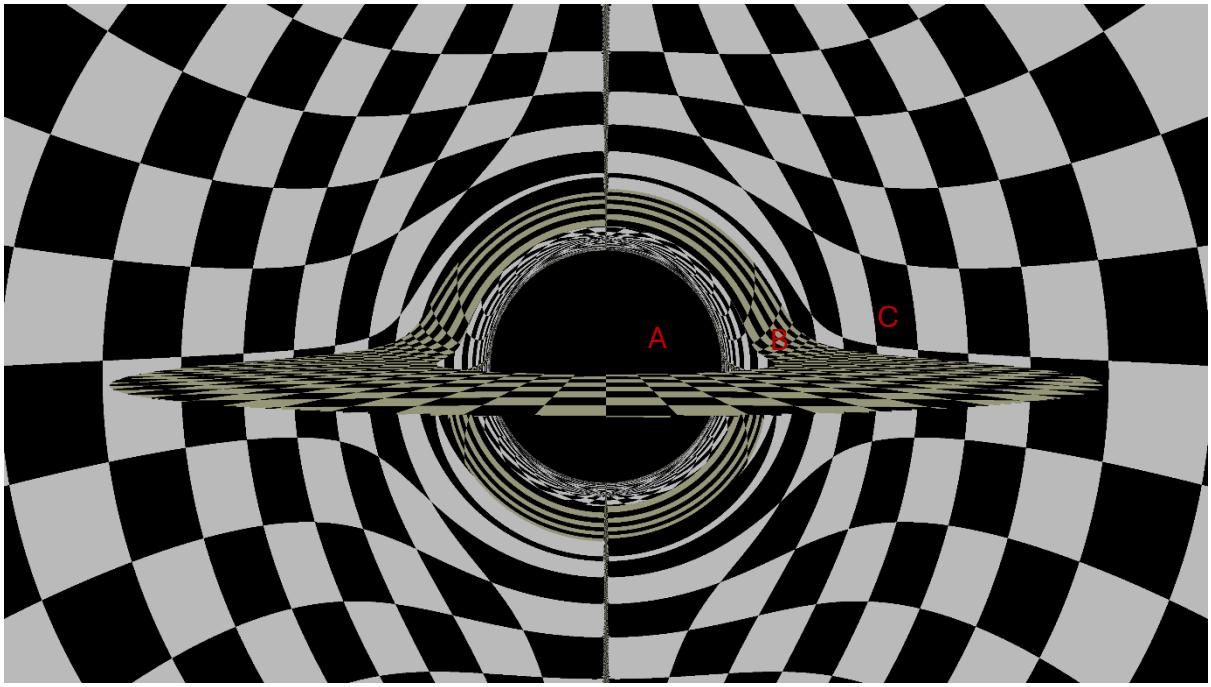


Figure 2: First render of a Schwarzschild black hole with checkerboard pattern on a planar accretion disk and surrounding space to showcase the distortion a black hole does to its geometry. Labelling: A – Perceived Black hole shadow, B – Accretion Disk, C – Lensed background pattern

In general relativity, all objects whether massive or massless follow trajectories through the four-dimensional manifold of spacetime. These trajectories are known as geodesics. The geodesic equation governs how these particles undergo free fall, going beyond Newton's equations by accounting for curved spacetime. It relates the components of

acceleration of a test particle to its velocities and thus its full trajectory can be numerically integrated and solved for. In the Kerr metric, which represents a rotating uncharged mass M, there is real curvature in the region surrounding the mass, thus even the trajectories of light rays are bent around it. The equations of motion of light rays in the Kerr metric are illustrated in Appendix B. (Carroll, 2004) [Figures 1,2]

It is possible for light rays to enter unstable orbits around the black hole. This is called the photon sphere, which encompasses a range of orbits around Kerr black holes, while only a single radius (r = 3M) is noted in a non-spinning Schwarzschild black hole. The photon sphere leads to gravitational lensing as light can orbit a black hole multiple times before escaping, and hence the same object can be seen from different perspectives. This can lead to the formation of Einstein rings when light rays emitted from an object behind a black hole are lensed around it. (Carroll, 2004) [Figure 3]

When a black hole is observed by an external observer, they do not perceive its event horizon (boundary beyond which even light rays cannot escape) directly. Since light rays can bend inwards, a larger number of them are caught by the event horizon, thus the shadow of the black hole, which is observed by an external observer, is much larger than its event horizon, about 2.6 times the horizon for a Schwarzschild black hole. (Carroll, 2004)

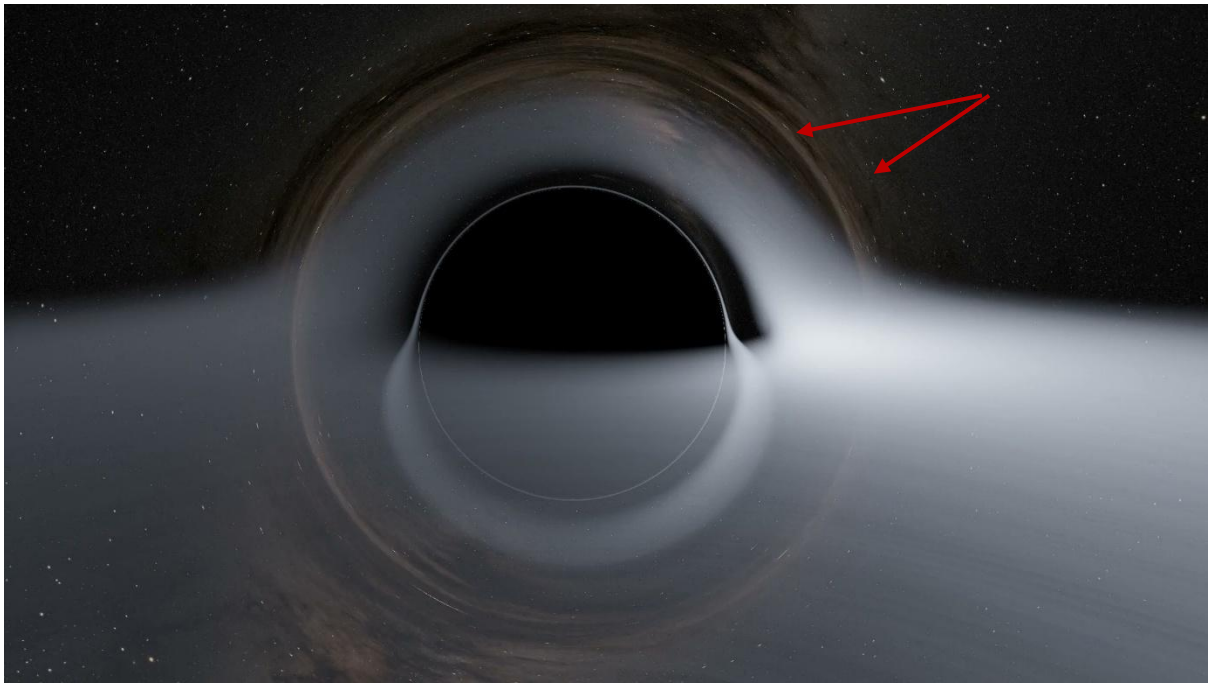


Figure 3: Up close render of a Kerr black hole with the Milky way behind forming a prominent Einstein ring which is due to the lensing effects of the black hole on the light emitted from the Milky way in the background.

A real accretion disk around a black hole consists of fiery ionized gas orbiting at high speeds resulting in high turbulence and magnetic fields. This is difficult to simulate

numerically, so it is often taken to be a smooth geometrically flat disk such as the Novikov-Thorne model in simulations (Novikov and Thorne, 1973).

2. Project Aims and Planning

2.1 Aim and success criteria

The aim of this project is to build a scientifically accurate Kerr metric ray tracer from scratch in Python with GPU acceleration and an optically dense Novikov Thorne accretion disk.

I will have achieved my aim if:

- Non-trivial conserved quantities along the null geodesics are preserved to 10^{-3} tolerance during numerical integration.
- The code runs in quick time – under 3 seconds per frame for high resolution e.g. 1080p or 4K.
- The resulting black hole shadow matches the analytical curve to within 10^{-2} tolerance across the range of spins.

2.2 Objectives and milestones

- The code can run and output an accurate render of a Kerr hole shadow. (26/04/26)
- The Novikov-Thorne volumetric accretion disk is implemented and photons passing through it can gain intensity. (05/05/26)
- Addition of relativistic effects such as kinematic doppler shift and gravitational redshift (08/05/26)
- Encoding GPU acceleration into the code to enable parallel processing of photons, speeding up execution time. (15/05/26)
- Allowing the camera to undergo a parametric trajectory and record a video from a series of frames and fixing bugs, the final objective. (01/06/26)

2.3 Approaches considered & Plan

Underlying framework of the program: C++ tends to be more efficient for low level programs whereas Python is known to be more versatile with additional libraries (Sharma, 2023). I chose Python for its versatility as well as my familiarity with the language so that I would not have to lose time looking up basic syntax thus the project could be approached more efficiently. Additionally, the processing of the program could not be done in regular Python due to its extreme inefficiency for highly computational codes. Among my options were Numba, which parallelises code across CPU cores achieving a moderate performance increase and is relatively easier to convert from

regular Python code; Taichi, which can efficiently parallelise code across GPU cores achieving a significant performance boost although its syntax is more complicated; and CUDA code, which achieves the maximum performance boost by optimising the memory allocation across CUDA GPU cores to precisely fit the code, though manually handling GPU memory can be tedious and features highly technical syntax. Initially after developing the base of my code, I opted for Numba as it would be simpler to convert the code and later implemented Taichi over CUDA as the performance difference is minuscule compared to the extra memory management required. Taichi also robustly allows switching the underlying framework, making the code more portable (Hu et al, 2019).

Why I chose to simulate Schwarzschild vs Kerr black holes: Although the Kerr metric is more complex and requires more computational power, it also encodes many striking effects such as frame dragging, shadow asymmetry and Doppler beaming (Kerr, 1963). Hence, it could also be of greater educational value for high school students. [Figures 4,5]

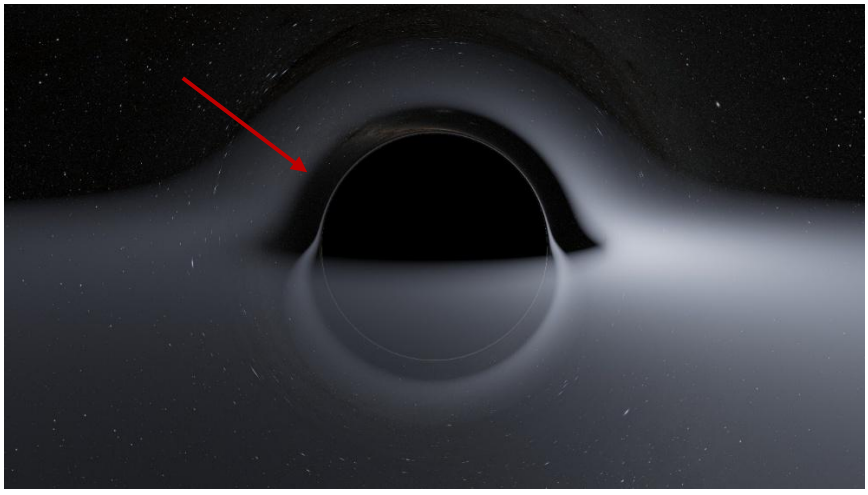


Figure 4: Up close render of a Schwarzschild (static) black hole

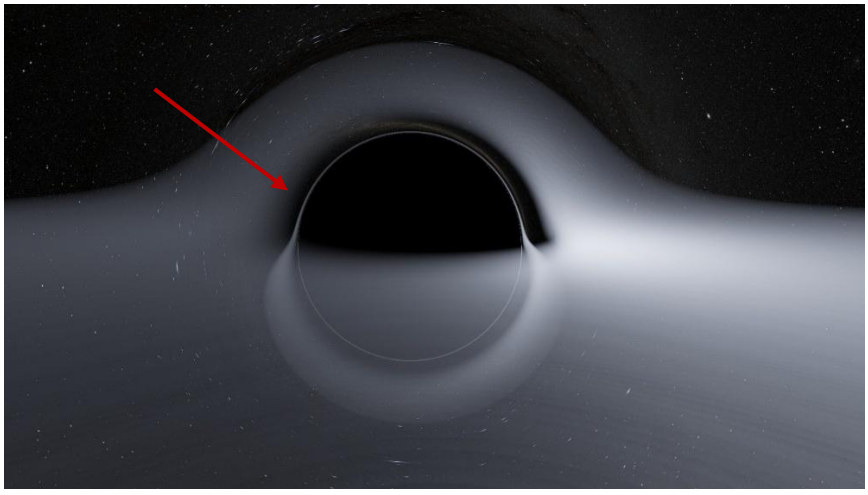


Figure 5: Up close render of a Kerr (spinning) black hole. Note the smaller inner boundary of the accretion disk and the slight tilt due to frame dragging compared to the Schwarzschild black hole

Numerical integration RK-45 vs RK-4: I started off using RK-4 as it is a simple algorithm to implement in code, however I later switched to RK-45 as its adaptive step sizing increases the efficiency as well as the accuracy particularly near high curvature regions (Dormand and Prince, 1980).

Real-time rendering vs offline video render: Although a real-time renderer could be more interactive and captivating to an audience, my code could only run at ~1-2 seconds per frame which is not fast enough for a 60 FPS real-time simulation. Perhaps if I could optimise my code further in the future I could switch to a real-time renderer.

The final timeline and step-by-step plan for execution of this approach is detailed in Appendix A.

2.4 Materials and people

- Processing: I used my home laptop with a NVIDIA GTX 1650 GPU to conduct the simulations. Although slightly dated, it was still capable of carrying them out in good time.
- Taichi framework's documentation: It was particularly useful for understanding Taichi syntax and debugging runtime errors.
- HDR sky map used: NASA Deep Star Maps 2020, I wanted to use a faithful star map of the Milky Way, so I used NASA's map.
- Alessandro Roussel's publicly available relativistic simulations including on the Kerr metric were used as inspiration to incorporate additional features and as a reference I could compare my renders against, although I did not read or adapt any of the source code.

3. Methodology

3.1 Code architecture

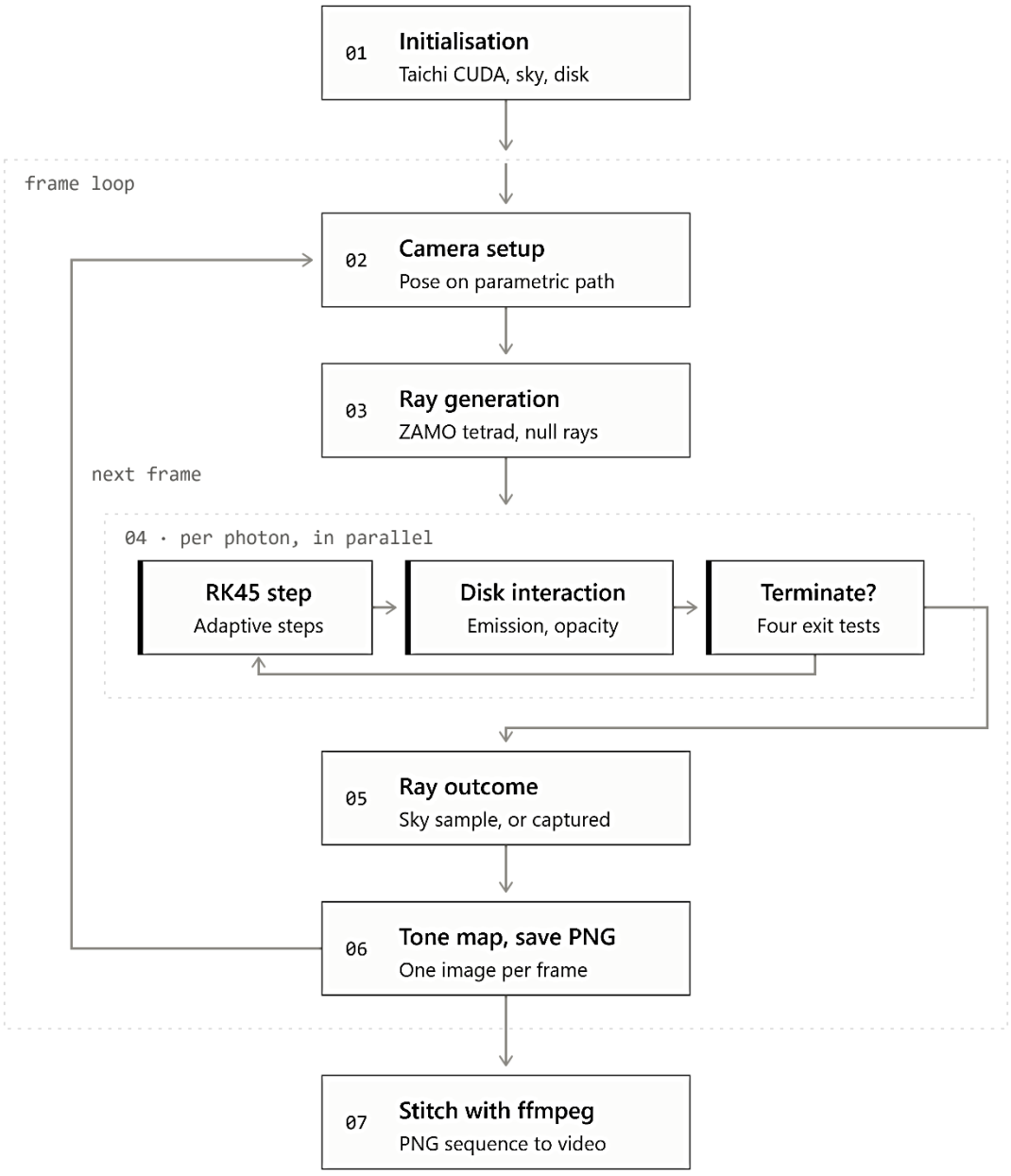


Figure 6: Architectural flowchart of the rendering pipeline.

3.2 Equations of Motion

The equations of motion used were the second order null geodesic equation in the Kerr metric using Boyer-Lindquist coordinates and parameterised by Mino time.

The geodesic equation is the standard for determining the trajectories of particles in curved spacetimes. The null equation is used for photons.

The use of second order equations (for theta and radial components) was motivated by the existence of turning points in these quantities, which would have to be manually checked, and sign flipped if using first-order equations. Second order equations naturally account for this. The equation for phi was first order as there are no turning points.

Mino Time $d\lambda = \Sigma^{-1} d\tau$ was used as the integration variable over the standard affine parameter as it absorbs Σ factor into the integrator, eliminating step sizing issues near the equatorial plane and allowing integration to proceed smoothly. It also completely decouples the equations of motion for radial and polar coordinates, allowing derivatives of the quantities to be expressed only in terms of those quantities, with no cross terms. As a result, the code in the future could use the equations solved analytically with canonical elliptic integrals.

Boyer-Lindquist coordinates are the standard for Kerr spacetime. It was preferred over alternatives like Cartesian Kerr-Schild, which do eliminate the spherical singularity at $\theta=0$, because Boyer-Lindquist maximises the polar symmetry of the Kerr metric, simplifying the equations of motion compared to Kerr-Schild. The result is that Boyer-Lindquist is much more time efficient compared to Kerr-Schild, hence it was chosen for this simulation.

3.3 Numerical Integration

Numerical Integration of geodesic trajectories is carried out by the Dormand Prince (RK-45) method. The Butcher Tableau of coefficients used when implementing RK-45 in code is illustrated in Appendix C.

Initially, it was carried out by the Runge-Kutta 4 (RK-4) method with a fourth order approximation, however it was not time efficient and resulted in coordinate instabilities at the pole where the ODE was stiff. This was because the fixed step size meant that photons passing near the pole would often jump over regions where the derivatives change rapidly, leading to it overshooting trajectories and resulting in a line of artifacts at the pole. RK-45 fixed this by using an adaptive step size, allowing it to shrink near the pole and accurately handle the coordinate singularity. It works by calculating a 4th and 5th order approximation and calculating the error between them, if it is too large the step size is reduced, and if it is very small the step size can be increased. Since the Kerr metric is asymptotically flat, (at large distances the derivatives do not change much) it can afford

to increase the step size at such distances, saving time on derivatives calculated. [Figure 7,8]

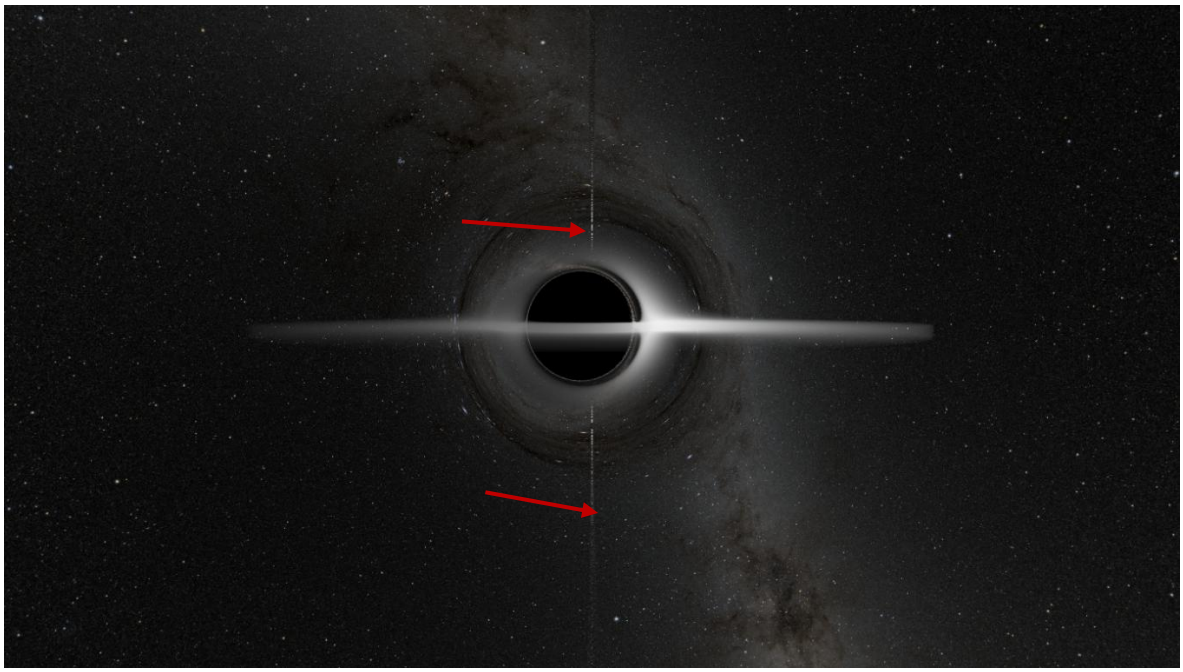


Figure 7: Original render with RK-4 with arrows pointing to the coordinate singularity at the pole (theta = 0) resulting in a line of defects.

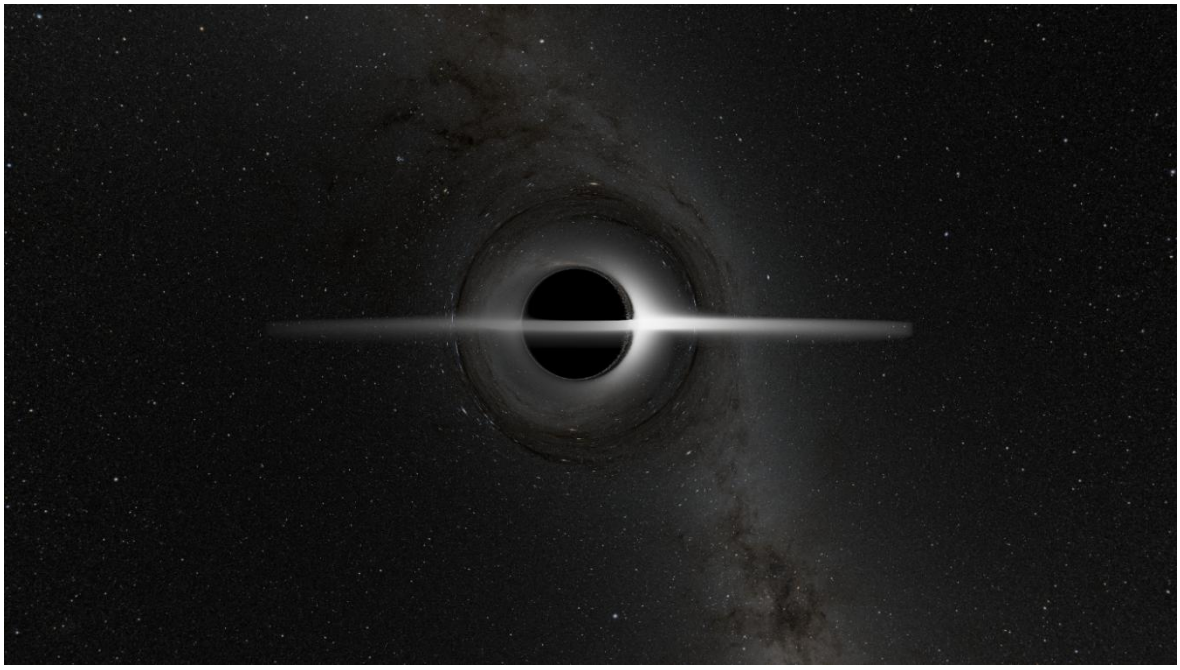


Figure 8: Improved render with RK-45 eliminating the coordinate singularity.

3.4 Accretion Disk model

The accretion disk utilised was a Novikov-Thorne accretion disk with vertical structure, the standard thin-disk model for simulations. It makes the assumptions of being geometrically thin, having a steady mass accretion rate and all heat generated by viscous friction radiated away locally. Although General Relativity Magnetohydrodynamical models (GRMHD) more accurately account for magnetic fields generated by the ionized gas moving at high speeds and the turbulence in accretion disks, they are substantially more computationally demanding, often requiring supercomputers to be simulated to scientific precision. As I was restricted to a home laptop, I opted for a thin-disk model. Among thin disk models, Novikov-Thorne was preferred over Shakura-Sunyaev as it accounts for relativistic correction factors for Kerr holes, such as there being an innermost stable circular orbit (ISCO) for orbiting matter unlike Newtonian gravity (Novikov and Thorne, 1973) (Shakura and Sunyaev, 1973).

Although the model itself was thin-disk, I decided to extend it to 3D by adding Gaussian vertical density $e^{-\frac{z^2}{2H^2}}$ to allow for volumetric integration. The Gaussian expression was used as it better represents hydrostatic equilibrium between the thermal gas pressure puffing the disk up and gravity crushing it down. The radial temperature of the disk was calculated using the mass accretion rate and relativistic factors $\mathcal{Q}(x,a), \mathcal{B}(x,a)\ and\ \mathcal{C}(x,a)$.

Volumetric ray marching was implemented by assigning Transmittance and Intensity values to each photon, which determined how much light had already been accumulated in the photon and the intensity of the light accumulated respectively. The radiative transfer equation was used to numerically calculate the photon's intensity as it passes through the optically thick disk, taking relativistic factors into account.

3.5 Relativistic Effects

For the accretion disk, the radial temperature included the Page-Thorne factors B, C and Q to account for frame dragging, spacetime curvature and stress-energy corrections (Page and Thorne, 1974).

Additionally, there is kinematic doppler shift near a Kerr hole as one side of the disk spins towards the camera at very high speed, blue-shifting light from it to higher energy, while the other recedes, red-shifting light to lower energy. There is also a deep gravitational well near the event horizon, so from the camera's perspective, the photons emerging from the innermost parts of the disk appear to lose energy climbing out of the gravitational well, resulting in gravitational redshift. All these effects were encompassed in the redshift factor $g = \frac{(k_\mu u^\mu)_{obs}}{(k_\mu u^\mu)_{emit}}$ calculated from photon four-momentum, calculated from metric

components and observer four-velocity. This was applied to the radiative transfer equation (Novikov and Thorne, 1973; Chandrasekhar, 1950).

3.6 Rendering

For photons escaping to infinity, an 8k HDRI/EXR star map was used to sample pixels for the background. Equirectangular projection was applied to the photon's position at infinity to determine the pixel coordinates. This is needed to accurately show how the trajectories of photons passing near the horizon is intensely warped, resulting in a lensed ring effect around the photon sphere known as a photon ring.

Additionally, anti-aliasing was also implemented by offsetting the initial velocities of each ray in 4 directions and independently integrating the 5 trajectories. The colour of the final pixel was then averaged out, resulting in a smoother final render without jagged edges.

After the pixels were generated, Reinhard tone mapping $L_{out} = \frac{L_{in}}{1+L_{in}}$ was implemented to compress the large HDR range of brightness values to a standard range [0,1] that can be displayed, followed by gamma correction $L_{out} = L_{in}^{1/2.2}$ to adjust the colour map to match human perception (Reinhard et al, 2002; Gonzalez and Woods, 2018).

3.7 Camera Initialisation

A standard 3D pinhole camera was used defined with an angular screen. Initially, a rectangular screen was used, preserving perspective, however it could not extend to field of views greater than 180°, so I switched to angular pixels allowing me to capture full 360° renders. The pixel coordinates for each photon are initialised by iterating over the normalised screen coordinates in the camera's basis. After this, a rotation matrix transforms each local direction vector from the camera basis to the absolute world basis. These vectors are projected onto a local reference frame (e_r, e_θ, e_ϕ) by taking dot products (James et al, 2015).

However, I noticed that whenever the camera came close to the event horizon, the screen would form a distinct partition with parts of the screen rendering completely different perspectives and this puzzled me for days. Eventually, I realised that the problem was in how the rays were initialised, assuming the camera to be stationary relative to an observer at infinity. Due to the frame dragging effects of the Kerr black hole, inside the ergosphere it is impossible for a camera to be stationary, thus it must instead be a Zero Angular Momentum Observer (ZAMO) following a free fall trajectory. Hence the radial, polar and azimuthal velocities had to be converted into conserved covariant momentum components p_μ for the equations to be physically meaningful inside the ergosphere (Bardeen et al, 1972) [Figures 9,10].

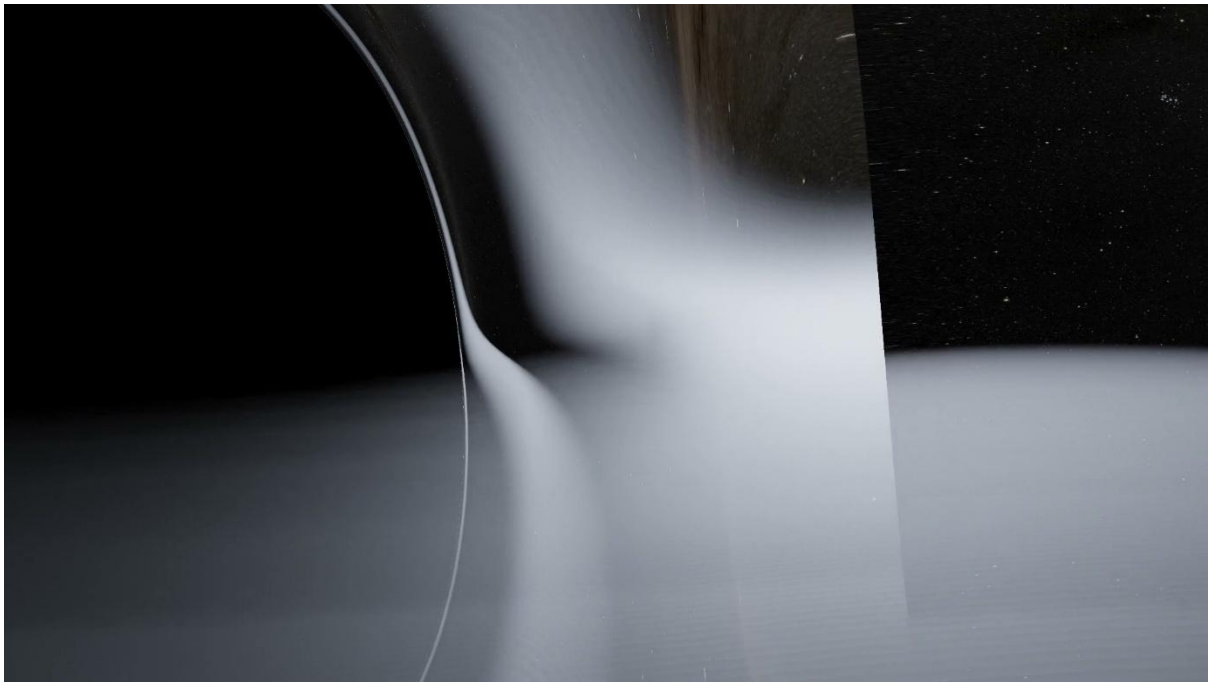


Figure 9: Original render without ZAMO tetrad initialisation of rays resulting in a distinct partition across the screen.

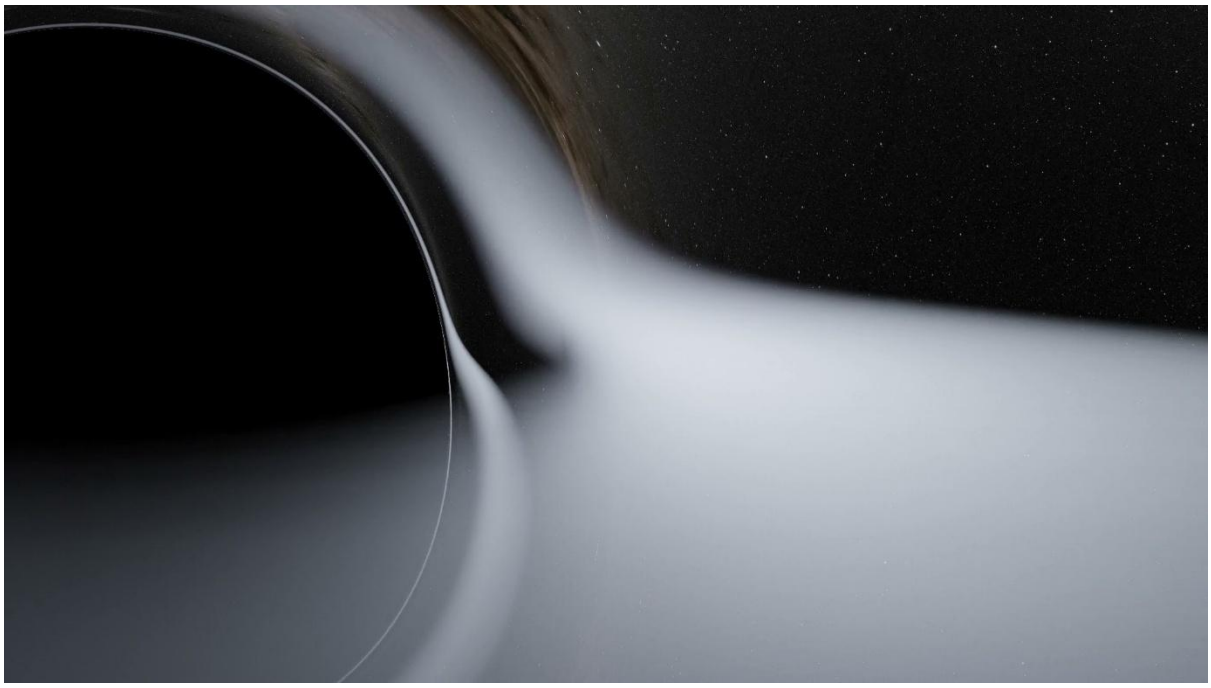


Figure 10: Fixed render with ZAMO tetrad initialisation removing the partition.

3.8 GPU acceleration and frame generation

The program is run from a Taichi kernel utilising the CUDA GPU cores of my laptop, and each frame is rendered individually as a PNG, later stitched together to form a video with the command-line tool FFmpeg. Originally, I started off writing this code with basic Python but realised that it was very time-inefficient to generate high-quality renders. Later, I switched to Numba to allow for parallel processing of the code across my CPU cores, significantly improving code performance. This is because each photon's trajectory can be numerically integrated completely independent of the others, making it an embarrassingly parallel algorithm. Regardless, each frame would take a considerable amount of time to render so I switched to GPU code, as the calculations involved in each step are simple enough to compute on a GPU core and can take advantage of the hundreds (896 to be exact in my computer's GPU) of GPU cores computing the same algorithm. I chose to implement it in Taichi as opposed to pure CUDA code, which allows for maximum control, as it enabled more high-level code without the cost of complex memory management.

For generating a video, the camera would be placed on a parametric trajectory with parameter 't' and its position and direction would be incremented between frames after each frame was saved as a PNG in sequential order. I used FFmpeg, a command-line tool, to quickly and efficiently stitch together the frames to form a MP4, each video file having a size of ~100-200 MB.

4. Results and Discussion

4.1 Validation against Theory

Drift in Conserved Quantities

In numerical simulations where numerical integrators are used, there is always accumulating error in the quantities calculated due to step size constraints and floating-point errors. Thus, a useful metric to judge an integrator's accuracy is to measure the drift in physically conserved quantities. For the Kerr metric, there are three main conserved physical quantities along a null geodesic: the Energy, the Z-Angular Momentum and the Carter Constant. However, I could not use these as a metric since my code operated by calculating these conserved quantities at the beginning and using them to integrate the equations of motion, naturally they would stay constant throughout. Thus, the error would have to be in terms of the position and momentum components. Since one of the first-order equations for motion was $p_r^2 = \left(\frac{dr}{d\lambda}\right)^2 = R(r)$ where R(r) is the radial potential, I chose to use the constraint $p_r^2 - R(r) = 0$ as a suitable drift metric.

For a set of 128 randomly sampled rays, I calculated the maximum relative drift of $p_r^2 - R(r)$ for each ray and plotted the median error and its interquartile range against the tolerance of the RK-45 integrator used using Matplotlib. A straight line of best fit was also plotted.

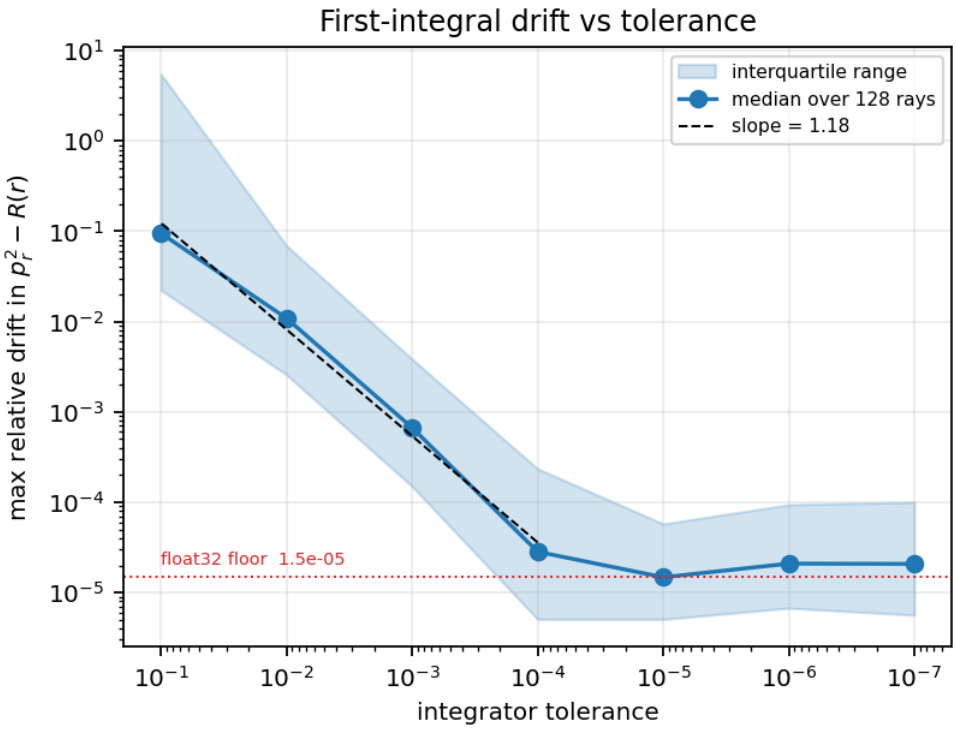


Figure 11: Comparison of maximum relative drift against tolerance of RK-45 integrator used

From the graph [Figure 11] we can see that maximum drift decreases linearly with integrator tolerance on a log-log plot up to around 1.5×10^{-5}. The decrease is expected as a tighter tolerance results in smaller step sizes taken by the integrator leading to smaller incremental drifts in the constraint. However, the numerical integration for the simulation was carried out with 32-bit floating-point numbers, so once the drift reaches near 1.5×10^{-5}, the limits of machine precision result in the error being unable to decrease as the additional precision cannot be stored in memory.

Comparison of Kerr hole shadow against Bardeen's Analytical Critical Curve

The trajectories of photons around the event horizon produce a shadow of the black hole that is larger than the event horizon. In the case of the Schwarzschild limit, it is a circle with impact parameter radius $b = 3\sqrt{3}\,M \approx 5.2M = 2.6r_s$. When the Kerr spin parameter is increased, there are additional effects on the boundary discussed in §4.3. This could be used to validate the render by comparing the simulation's shadow boundary against Bardeen's Curve (Bardeen, 1973).

To simulate this, a bisection search was used to numerically find the shadow boundary by shooting photons at varying angles which affects their impact parameters, and testing

whether they escape to infinity or get captured. For simplicity, the camera was placed on the equatorial plane facing the Kerr hole. Then, the angle separating an escaping ray and a captured ray is determined. Once a rough estimate is obtained it is improved iteratively up to a limit (10^{-4}). From the critical impact parameters, the angles α and β are determined and each point plotted.

For the analytic curve, a dense array of 200000 points of 'r' are sampled between the maximum and minimum photon orbit radii and the impact parameters (ξ, η) for these points are calculated analytically. The physically invalid points (η < 0) are taken out. The angles are again determined from impact parameters and plotted to form a closed loop.

The simulation's deviation from the analytic curve $|b_{measured} - b_{bardeen}|$ was computed and plotted along with its variation against the spin parameter 'a'.

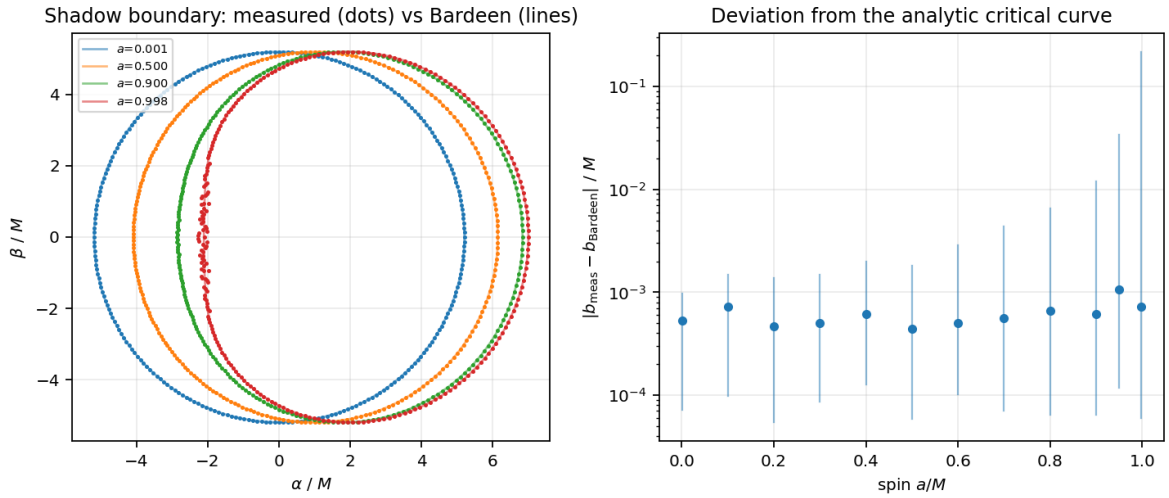


Figure 12: Comparison of the Black Hole shadow boundary from the simulation against the analytical Bardeen Critical curve and its deviation

From the graphs [Figure 12], it is evident that the curves line up nicely with the deviation staying under 10^{-3} for the most part. However, for the highest spin parameter (0.998) the points are poorly determined on the prograde (left) edge. This can be attributed to the fact that the prograde photon orbit lies much closer to the horizon, and as the spin parameter approaches 1, the coordinate separation of the photon orbit and event horizon approaches 0. Since there is a coordinate singularity in the Boyer-Lindquist equations of motion at the event horizon, the points are scattered by the stiffness of the ODEs at that edge. This results in a larger variation of the deviation at the point closest to a = 1.

4.2 Performance

GPU vs CPU

To determine the speedup factor in frame renders when utilising GPU CUDA cores as opposed to CPU cores, a graph of the render time was plotted against number of pixels per frame for both CPU and GPU code in a log-log plot, by using the same kernel but changing the architecture used to run the code (ti.cpu and ti.cuda).

The GPU speedup factor = CPU time / GPU time was also calculated and plotted as a bar chart against pixels per frame. In Figure 9, there are only three CPU points as the run was unfeasibly slow at the largest size.

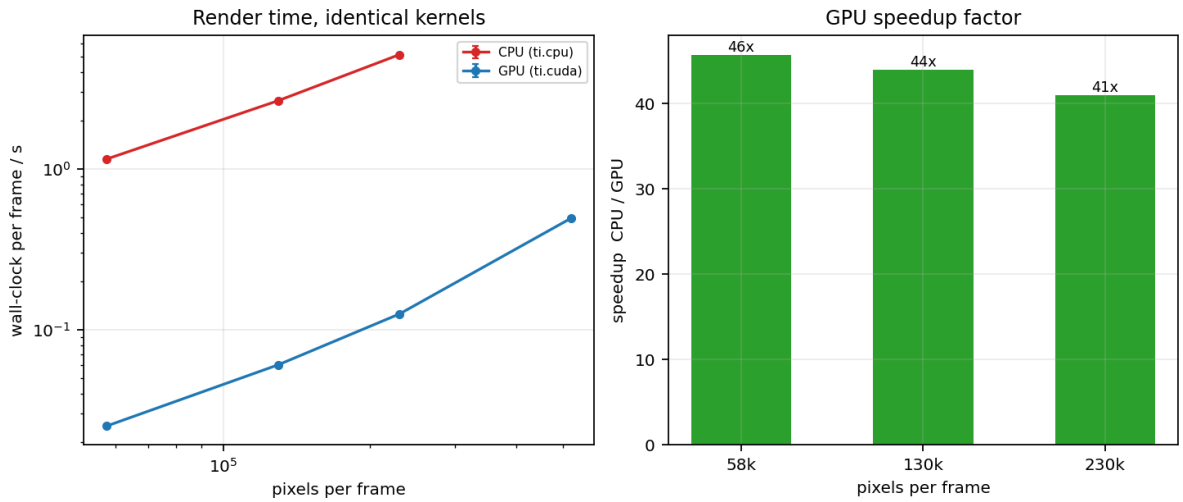


Figure 13: Comparison of CPU to GPU code render times and the GPU speedup factor against image resolution

From the plots [Figure 13], GPU code has a significantly lesser render time for a given frame than CPU code. Both the codes scale proportionally with pixels on a log-log plot as doubling the number of photons would require double the amount of time once all the cores are being utilised. The GPU time is significantly lower due to its faster processing speed and greater number of cores. However, the speedup factor decreases with pixels as both plots scale linearly once all cores are saturated, so the performance factor gets normalised by the increasing render time.

RK-4 vs RK-45

I wanted to establish the cost over evaluations for the fixed step RK-4 and adaptive step RK-45 integration methods. I did this by plotting the worst-case drift in $p_r^2 - R(r)$ over the number of derivative evaluations for a fan of rays, using RK-45 or RK-4. The tolerance of RK-45 or the step size for RK-4 was altered to produce a line of points. However, not all rays were at the optimal error/evaluations: if their tolerance exceeded the float 32 limit, the drift would remain the same. Hence, a Pareto function was used to filter out the

suboptimal points (hollow circles) while the rest were plotted on a line (solid points) in a log-log plot.

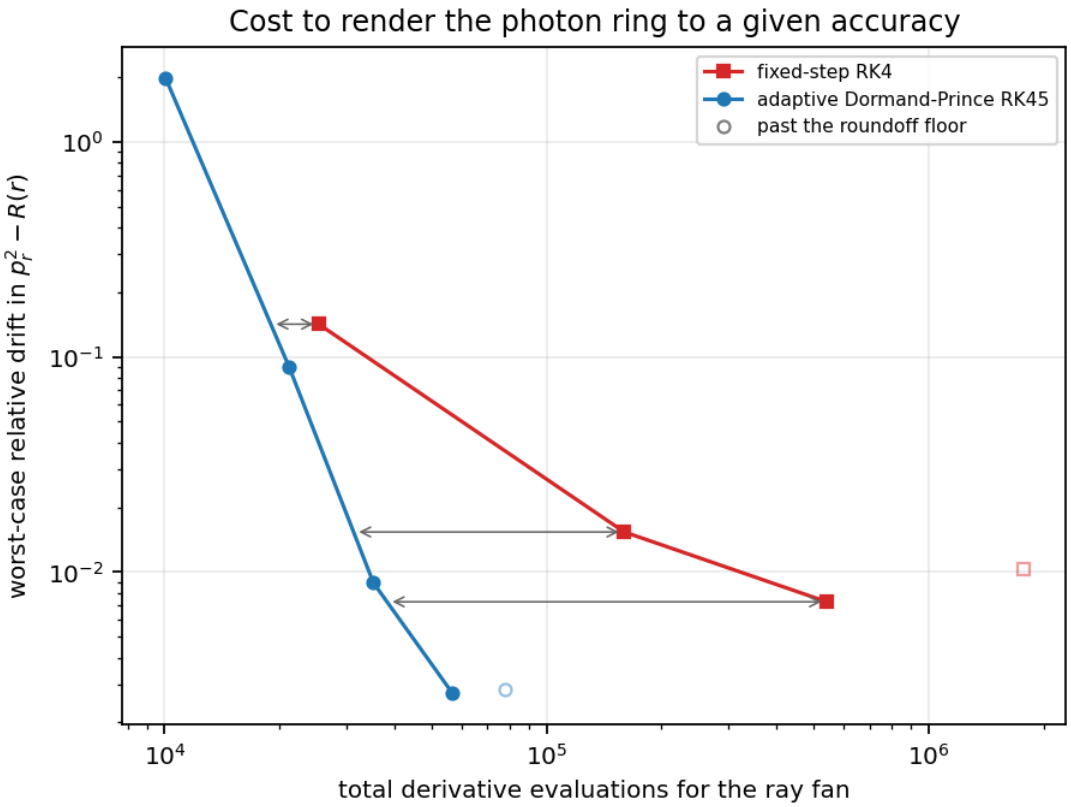


Figure 14: Comparison of RK-4 vs RK-45 worst case drift against derivative evaluations

From the plot [Figure 14], it is evident that RK-45 requires vastly fewer derivative evaluations compared to RK-4 for the same drift, indicated by the grey arrows in the graph.

4.3 Parameter Study

From my simulation, I sought to answer a question: "How does the observed shadow asymmetry scale with spin a?"

From literature, I was aware of three interesting properties of the shadow that scaled with spin parameter: The centroid displacement, shadow flattening and shadow area. I reused the data obtained from comparing the Bardeen analytic curve to my simulation for these calculations.

Firstly, for both sets of points (α, β) I calculated three geometric metrics using the surveyor's shoelace formula: the area enclosed by the polygon $A = \frac{1}{2}\sum(\alpha_i\,\beta_{i+1} -$

$\alpha_{i+1}\ \beta_i)$, then the equivalent radius for a perfect circle of same area $r_{eq} = \sqrt{\frac{A}{\pi}}$ and the centroid (α_c, β_c) of the polygon using the formula:

$$C_x = \frac{1}{6A}\sum_{i=0}^{n-1}(x_i + x_{i+1})(x_i y_{i+1} - x_{i+1} y_i)$$

Then the asymmetry metrics were calculated from these as follows:

- Centroid displacement: $\frac{\alpha_c}{R_{eq}}$ Due to the vertical symmetry β_c would equate to 0, so α_c was only considered and normalised by the equivalent radius.
- Flattening: $(b_{98\%} - b_{2\%})/\bar{b}$. The distance of each point to the centroid b_i was computed. Flattening is usually calculated by max radius – min radius, however the 98th and 2nd percentile were used to avoid outlier points that disrupt the calculation. The value was normalised by the mean radius
- Shadow Area: Plotted directly

The results were plotted with the numerical quantities in points and the analytic quantities on a curve.

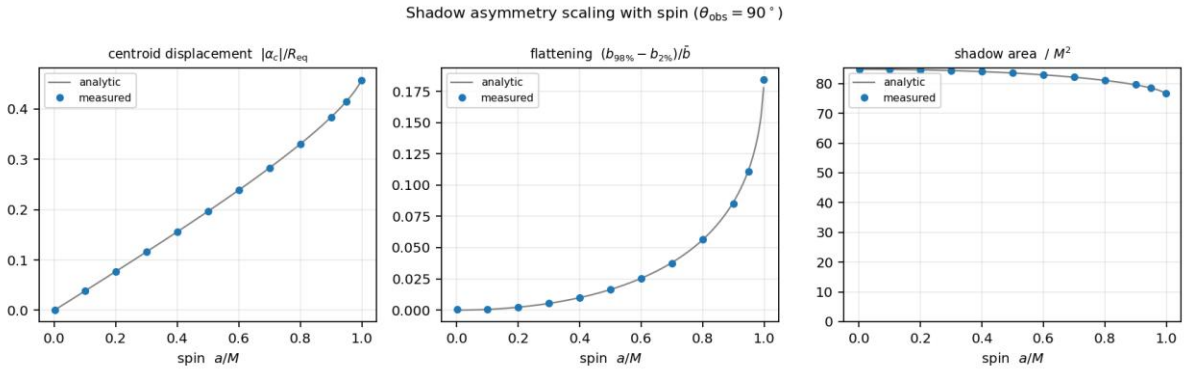


Figure 15: Case study of the shadow asymmetry (centroid displacement, curve flattening and shadow area) scaling with spin parameter

The results show that the numerical and analytic results match up nicely, as already illustrated in the Bardeen Curve plot.[Figure 15].

The centroid displacement increases linearly with spin parameter, with the Schwarzschild black hole having no displacement. This can be attributed to the increased effect of frame dragging as spin increases, photons on the prograde side can travel much closer to the event horizon whereas they orbit at larger distances on the retrograde side. The result is that the centroid of the shadow gets shifted.

For the flattening, the shadow starts circular, but flatness increases considerably for very high spins 'a' > 0.8. This can be attributed to two competing effects: Firstly, the frame dragging results in the prograde photon orbit approaching the event horizon as the 'a' parameter approaches 1, pulling the orbit into a prolate shape. Secondly, the spinning

black hole has an oblate spheroid shape, bulging at the equator which tends to push the orbit into an oblate shape. These two effects mostly cancel out for lower spin, but as the spin parameter approaches 1, the frame dragging effect overpowers the bulging resulting in the spike (Psaltis et al, 2021).

The shadow area reduces by only 9.6% over the range of spin parameters. This counterintuitive result can be explained by the prograde edge shrinking and the retrograde edge expanding, which distorts the shape of the shadow but barely affects its area. The reduction in capture area on the prograde side is almost completely balanced by the increase in area on the retrograde side. Thus, the total shadow area of a black hole depends on its mass, but only weakly on its spin.

5. Conclusion

5.1 Were the aims met?

- Conserved quantities along the null geodesics are preserved to **10^{-4} tolerance**
- The code runs in quick time **– under 1 second** per frame for 4K resolution using CUDA cores.
- The resulting black hole shadow matches the analytical curve with a **median deviation of 10^{-3}** for $a \leq 0.95$.

The code passes all the aims I originally laid out for the most part. There are still some aspects to consider further such as the deviation of the shadow from analytic results for very high spin.

5.2 Limitations

There were a few limitations on the physical accuracy of the simulation:

- The accretion disk was modelled on a geometrically thin and uniformly smooth Novikov-Thorne disk. However real accretion disks contain hot ionized plasma that forms violent turbulent spirals and produces magnetic fields affecting its mass distribution. These would be better modelled by GRMHD simulations.
- Moreover, the volumetric integration through the accretion disk only accounted for primary radiation and did not include return-radiation or secondary-emission effects.
- The gravitational redshift I implemented was solely based on intensity and not colour, even though the frequency of the photons would change. This was because the peak wavelength of light emitted by the accretion disk lies in X-ray band for stellar-mass holes, which is not visible, thus I felt it would not make physical sense to alter redness or blueness depending on the redshift.

- I chose 32-bit floating point precision (float32) for all numerical values rather than 64-bit precision (float64) as theoretically the FP64:FP32 hardware ratio of my GPU is 1:32, so float32 is significantly faster, although the numerical precision was reduced, as illustrated in the 1.5×10^{-5} lower bound on the numerical error.

5.3 Further work

I could switch the code to utilise Kerr-Schild Cartesian coordinates rather than Boyer-Lindquist as these have no coordinate singularity and would allow the camera to venture past the event horizon.

Additionally, I could focus on optimising the code to generate frames at a fast enough speed to enable a real-time simulation rather than video generation.

Furthermore, I could encode more relativistic effects such as aberration when the camera itself moves at a high speed to increase physical accuracy. If my computer allows it, I could try running a very simple GRMHD accretion disk simulation.

5.4 Implications for the wider world

The simulation I made could be used to teach general relativity to a public audience. I was able to show this at my school's STEM club and alongside it, I explained interesting topics in relativity such as gravity as curvature in a geometry, the bending of light and lensing effects, and relativistic redshifts.

While my current simulation matches literature in expected event horizon shadow and accretion disk shape (Luminet, 1979; Novikov and Thorne, 1973), it is still far from being comparable to real black holes with violent accretion disks and fiery plasma jets.

The code is runnable on home computers with a moderate GPU, and the architecture can be changed easily with Taichi settings before running the code. Thus, it is accessible to anyone who wishes to explore rotating black holes further.

6. Reflection

This project enabled me to study the physics of General relativity including tensors and differential geometry, the engineering of numerically solving differential equations and the computer science of ray tracing programs. I was able to grasp the connection between all these fields and understand that together they play an important role in research today. It was quite challenging to understand all the theoretical ideas utilised in the program, especially the mathematical concepts, and it took me several months of prior study and research before I attempted this project, but I am glad to have learnt from a variety of fields and hope to continue learning and improving my simulation.

7. Ethics, safety and authenticity

7.1 Scientific communication ethics and safety

It is important to disambiguate scientific rendering and artistic effects. This is because physically styled images can often be mistaken for real scientific research and circulated. In my code, the render of the Kerr black hole was calculated by integrating the path of photons with the known physical equations of motion. The equations for relativistic redshift were taken from primary literature. However, I applied Reinhard tone mapping and gamma correction after-effects on the obtained image to make it more discernible to the human eye as an artistic choice.

In terms of safety, this was a purely computational project involving no physical hazards, human participants or personal data. The only notable considerations were screen time and the thermal load of my laptop during GPU renders.

7.2 Licensing and Attribution

HDR sky map used: NASA/Goddard Space Flight Center Scientific Visualisation Studio. Gaia DR2: ESA/Gaia/DPAC. Constellation figures based on those developed for the IAU by Alan MacRobert of Sky and Telescope magazine (Roger Sinnott and Rick Fienberg).

Licenses of python libraries and tools used:

- NumPy: BSD License
- Taichi: Apache License 2.0
- OpenCV: Apache License 2.0
- FFmpeg: LGPLv2.1

7.3 Environmental cost

The energy consumed from GPU rendering can be determined from power x render time. For a Nvidia GTX 1650 GPU on a laptop, the power usage is 50 W. Each rendered frame takes up to a second so the energy can be estimated at 50 J. For a 30 second, 30 FPS video (900 frames), the energy consumed is 45 kJ. This is a meagre amount of energy and can be compared to running a 60-watt incandescent bulb for 12.5 minutes or boiling roughly half a cup of water in a kettle.

7.4 Authenticity and tool disclosure

The ray tracing code was written fully by me, and I used libraries including Taichi to facilitate conversion into GPU code, NumPy to handle arithmetic outside the GPU and OpenCV to convert frames into PNG renders.

The use of AI tools (Perplexity) was limited to obtaining obscure formulae from technical papers, namely the kinematic Doppler and gravitational redshift functions, later verified from their primary sources and for interpreting and identifying Taichi runtime errors.

8. References

Bardeen, J.M., Press, W.H. and Teukolsky, S.A. (1972) 'Rotating black holes: locally nonrotating frames, energy extraction, and scalar synchrotron radiation', The Astrophysical Journal, 178, pp. 347–369. (Accessed: 6 August).

Bardeen, J.M. (1973) 'Timelike and null geodesics in the Kerr metric', in DeWitt, C. and DeWitt, B.S. (eds.) Black Holes (Les Astres Occlus). New York: Gordon and Breach, pp. 215–239. (Accessed: 5 May).

Carroll, S.M. (2004) Spacetime and geometry: an introduction to general relativity. San Francisco: Addison-Wesley. (Accessed: 7 April).

Chandrasekhar, S. (1950) Radiative Transfer. Oxford: Clarendon Press. (Accessed: 15 May).

Deep Star Maps (2020) NASA/Goddard Space Flight Center Scientific Visualisation Studio. Gaia DR2: ESA/Gaia/DPAC. Constellation figures based on those developed for the IAU by Alan MacRobert of Sky and Telescope magazine (Roger Sinnott and Rick Fienberg).

Dormand, J.R. and Prince, P.J. (1980) 'A family of embedded Runge-Kutta formulae', Journal of Computational and Applied Mathematics, 6(1), pp. 19–26. (Accessed: 15 April).

Event Horizon Telescope Collaboration (2019) 'First M87 Event Horizon Telescope Results. I. The shadow of the supermassive black hole', The Astrophysical Journal Letters, 875(1), p. L1. (Accessed: 1 May).

Event Horizon Telescope Collaboration (2022) 'First Sagittarius A* Event Horizon Telescope Results. I. The Galactic Center supermassive black hole', The Astrophysical Journal Letters, 930(2), p. L12. (Accessed: 2 May).

Gonzalez, R.C. and Woods, R.E. (2018) Digital Image Processing. 4th edition. New York: Pearson. (Accessed: 20 May).

Hu, Y., Li, T.-M., Anderson, L., Ragan-Kelley, J. and Durand, F. (2019) 'Taichi: a language for high-performance computation on spatially sparse data structures', ACM Transactions on Graphics, 38(6), Article 201. (Accessed: 22 April).

James, O., von Tunzelmann, E., Franklin, P. and Thorne, K.S. (2015) 'Gravitational lensing by spinning black holes in astrophysics, and in the movie Interstellar', Classical and Quantum Gravity, 32(6), p. 065001. (Accessed: 3 April).

Kerr, R.P. (1963) 'Gravitational field of a spinning mass as an example of algebraically special metrics', Physical Review Letters, 11(5), pp. 237–238. (Accessed: 20 April).

Luminet, J.-P. (1979) 'Image of a spherical black hole with thin accretion disk', Astronomy and Astrophysics, 75(1–2), pp. 228–235. (Accessed: 24 April).

Novikov, I.D. and Thorne, K.S. (1973) 'Astrophysics of black holes', in DeWitt, C. and DeWitt, B.S. (eds.) Black Holes (Les Astres Occlus). New York: Gordon and Breach, pp. 343–450. (Accessed: 30 April).

Page, D.N. and Thorne, K.S. (1974) 'Disk-accretion onto a black hole. Time-averaged structure of accretion disk', *The Astrophysical Journal*, **191**, pp. 499–506. doi: 10.1086/152990.

Psaltis, D., Talbot, C., Payne, E. and Mandel, I. (2021) 'Probing the black hole metric. I. Black hole shadows and binary black-hole inspirals', Physical Review D, 103(10), article 104036. Available at: https://doi.org/10.1103/PhysRevD.103.104036. (Accessed: 2 August).

Reinhard, E., Stark, M., Shirley, P. and Ferwerda, J.A. (2002) 'Photographic tone reproduction for digital images', ACM Transactions on Graphics, 21(3), pp. 267–276. (Accessed: 25 May).

Roussel, A. (2020) 360° VR Simulation #2 of a Black Hole. YouTube video, 18 February. Available at: https://www.youtube.com/watch?v=i0HLZRealAY (Accessed: 8 April).

Shakura, N.I. and Sunyaev, R.A. (1973) 'Black holes in binary systems. Observational appearance,' *Astronomy and Astrophysics*, **24**, pp. 337–355.

Sharma, A. (2023) 'C++ vs. Python: A Performance Comparison using an Example', C# Corner. Available at: https://www.c-sharpcorner.com/article/cpp-vs-python-a-performance-comparison-using-an-example/ (Accessed: 7 August).

TOI World Desk (2024) ' What happens when you fall into a black hole?' The Times of India, May 7. Available at: https://timesofindia.indiatimes.com/science/new-nasa-visualisation-takes-viewers-beyond-the-brink-of-a-black-hole/articleshow/109911930.cms (Accessed: 7 August).

9. Glossary

General relativity	A theory that treats gravity as curvature of spacetime rather than a force.
Geodesic	The path a freely falling object follows through curved spacetime. A null geodesic is the path of a photon.
Metric	mathematical object that encodes distances and curvature in a specific spacetime. Kerr and Schwarzschild black holes are represented by their corresponding metrics.
Spin parameter (a)	dimensionless quantity signifying the spin of a Black hole. Goes from 0 (stationary) to 1 (maximal spin).
Event horizon	boundary beyond which nothing escapes, even light.
Shadow / critical curve	the dark region seen by an external observer, around 2.6× larger than the event horizon.
Photon sphere / photon ring	specific radii where light can orbit the hole, producing the bright lensed ring.
Gravitational lensing	the bending of light by spacetime curvature, allowing one object to be seen from several perspectives.
Frame dragging	a rotating mass drags local spacetime with it as it rotates.
Ergosphere	A region outside the horizon where frame dragging makes remaining stationary relative to infinity impossible.
Prograde / retrograde	rotating with / counterrotating against the spin of the black hole.
ISCO	innermost stable circular orbit, the closest radius matter can stably orbit a black hole.
Accretion disk	a rotating disk of hot plasma that accretes around a black hole and slowly spirals in.
Novikov- Thorne model	The simplest relativistic thin-disk model used in my simulation
GRMHD	general relativistic magnetohydrodynamics. Advanced accretion disk model accounting for the turbulent and magnetic effects of plasma.
Radiative transfer equation	Determines how intensity of light changes as it passes absorbing and emitting material.
Kinematic doppler shift	Matter moving towards/away from an observer appears brighter/darker as its photons get redshifted.
Gravitational redshift	Photons lose energy as they climb out of the black hole's gravitational well.
RK-4/RK-45 (Dormand Prince)	Runge-Kutta 4 is a fixed-step numerical integration method, whereas RK-45 compares 4th and 5th order error estimates to control error.
Float32/64	Floating point numbers stored in computer with either 32 bits or 64 bits of memory.
Embarrassingly parallel	A problem which can be split into completely independent tasks, in this case the integration of each individual photon's path.
CUDA	Nvidia's platform for running code across GPU cores.

10. Appendices

Appendix A: Project Timeline

Date	Event
20/04/26	I started with the ray tracing base
21/04/26	Added solving the geodesic equation in spherical Boyer Lindquist coordinates with RK-4
23/04/26	I decided to switch to Mino time in my numerical integration as it makes the equations separable and accounts for higher precision near the event horizon
24/04/26	Changed the equations of motion from first order to 2nd order BL to handle turning points effectively (accounts for relative maximums/minimums in orbits)
25/04/26	Tried my first attempted render with three shades for black hole shadow, accretion disk and background
26/04/26	After a few tweaks I was able to get the shape of the shadow and disk to match up what I had seen in literature
26/04/26	There was considerable aliasing making the accretion disk look ugly, so I implemented anti-aliasing to smoothen it out
28/04/26	Added checkerboard texture mapping to disk and background. I was able to appreciate the extreme lensing effects of these black holes; it also matched with literature.
30/04/26	I switched from regular python code to Numba, as it would parallel the code across my CPUs, with the intention of setting up GPU code later.
01/05/26	I tried other things with the accretion disk to make it look more accurate, but it did not really work.
01/05/26	I was contemplating downloading an accretion map to serve as my disk, but I decided to use the Novikov-Thorne thin disk model myself.
03/05/26	I expanded the flat disk to 3d by adding a gaussian vertical distribution.
05/05/26	I added transparency and opacity to make accretion disk realistic with volumetric integration through it.
06/05/26	I implemented kinematic doppler beaming.
07/05/26	I started work on gravitational redshift.
08/05/26	Finished redshift, utilising accurate Kerr correction terms as opposed to the Newtonian approximation
09/05/26	The pole (theta = 0) was a big annoyance, and I wanted to get rid of it. Many other ray tracers I had seen online also had this pole but did not specify how to get rid of it.
10/05/26	I was considering switching to Cartesian Kerr- Schild coordinates that would not have the issue of the spherical pole, however those coords are also much more inefficient so I decided to keep BL for now.
12/05/26	Among the things I tried, I switched from RK-4 to RK-45 (Dormand Prince method), and this got rid of the pole finally. Due to its adaptive step sizing, it also resulted in more accuracy with less execution time.
13/05/26	Started converting the program into GPU code with Taichi

15/05/26	Finished conversion, it took a while and there were many hiccups as Taichi's syntax is extremely specific and is not compatible with NumPy, however there were dramatic improvements: originally, 4k renders took up to 20 min to make, but now they could be done in under 1 second
16/05/26	Added slight blue shift (plasma so unimaginably hot only X ray visible, perceived as blinding pure light) blueness conveys heat
17/05/26	Added accurate relativistic radial temperature function
18/05/26	Used smooth step function to smooth out abrupt edge of accretion disk
20/05/26	Finally added video renders by rendering a series of PNGs. It takes ~1 second per frame. I used FFmpeg to stitch outputted PNGs into mp4
21/05/26	camera moves on parametric trajectory <x(t), y(t),z(t)> that I can tinker with on Desmos
22/05/26	searching for interesting trajectories (added code adjusting FOV and camera direction if too close to EH so that it would not occupy the entire FOV)
23/05/26	Swapped flat screen camera ray tracing for spherical to allow FOV to extend much higher up to 360 degrees.
25/05/26	When the camera came close to the EH, there was a suspicious partition with parts of the screen rendering completely different perspectives and this puzzled me for days.
29/05/26	Eventually I found the solution to be the initialization of the null geodesics. I had to use the ZAMO tetrad.
30/05/26	Refinements: Added blueshift for distant stars when camera falling in, correct step size across the disk when photons pass through it for intensity calculations, and correct energy for photons based on their initial position
31/05/26	Finetuned the accretion disk artistic look according to suggestions from others
01/06/26	Added 360° video functionality

Planned vs Actual project timeframe

Section	Planned duration	Actual duration
Literature review of general relativity (tensors, Riemannian geometry, the geodesic equation, the Schwarzschild, and Kerr metrics), ray tracing and numerical integration methods.	1 month	2 months
Setup the ray tracing base and generate a simple monotone render of the Kerr black hole with accretion disk.	7 days	7 days
Convert the flat accretion disk to a volumetric one and implement volumetric integration of the photons.	4 days	10 days
Implement relativistic effects in the code (Kinematic doppler shift and gravitational redshift).	3 days	3 days
Convert the code from CPU to GPU architecture to speed up runtime.	3 days	7 days
Debug and finetune any issues with the rendering program.	3 days	14 days
Add video functionality by having a camera move on a parametric trajectory.	2 days	2 days
Render interesting trajectories in the Kerr metric and optionally add 360° video functionality.	3 days	3 days

Approximate total hours spent: 110 (45 on literature review, 65 on writing code)

Appendix B: Kerr metric and equations of motion

For a spinning mass M and angular momentum J in Boyer Lindquist coordinates (t, r, θ, ϕ), this can be described by the metric:

$$ds^2 = -\left(1 - \frac{r_s r}{\Sigma}\right) dt^2 + \frac{\Sigma}{\Delta} dr^2 + \Sigma d\theta^2 + \left(r^2 + a^2 + \frac{r_s r a^2}{\Sigma} \sin^2\theta\right) \sin^2\theta \, d\phi^2 - \frac{2 r_s r a \sin^2\theta}{\Sigma} dt \, d\phi$$

Where:

$$a = \frac{J}{Mc}$$

$$\Sigma = r^2 + a^2 \cos^2\theta$$

$$\Delta = r^2 - r_s r + a^2$$

$$r_s = 2M$$

In geometric units G = c = 1

Equations of motion* in Mino time λ:

$$\frac{d(p_r)}{d\lambda} = \frac{1}{2}\frac{dR}{dr}$$

$$\frac{d(p_\theta)}{d\lambda} = \frac{1}{2}\frac{d\Theta}{d\theta}$$

$$\frac{d\phi}{d\lambda} = \frac{a}{\Delta}(r^2 + a^2 - aL_z) - a + \frac{L_z}{\sin^2\theta}$$

Where R and Θ are the radial and polar potentials respectively.

$$R(r) = (r^2 + a^2 - aL_z)^2 - \Delta(Q + (L_z - a)^2)$$

$$\Theta(\theta) = Q + a^2 \cos^2\theta - L_z^2 \cot^2\theta$$

$E, Q \text{ and } L_z$ are the conserved Energy, Carter Constant and Axial Angular Momentum

*Note: The equation of motion for the time coordinate was omitted as the metric is time invariant so I chose not to integrate the time component of the photons for efficiency.

Appendix C: Butcher Tableau for Dormand Prince Numerical Integration

The Butcher Tableau is a convenient grid layout of the coefficients used in a Runge Kutta Integration method. The table below shows the coefficients used in my simulation when numerically integrated with the Dormand Prince (RK-45) method.

0							
1/5	1/5						
3/10	3/40	9/40					
4/5	44/45	-56/15	32/9				
8/9	19372/6561	-25360/2187	64448/6561	-212/729			
1	9017/3168	-355/33	46732/5247	49/176	-5103/18656		
1	35/384	0	500/1113	125/192	-2187/6784	11/84	
	35/384	**0**	**500/1113**	**125/192**	**-2187/6784**	**11/84**	**0**
	5179/57600	**0**	**7571/16695**	**393/640**	**-92097/339200**	**187/2100**	**1/40**